

Build Task — Reusable Chain Module — Hints & Solution

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 3 depth levels: **Basic** (the plain idea), **Intermediate** (proper Python), **Advanced** (how a real codebase would actually write it).

- [Hint 1 — What you're building, and the exact pieces](#)
- [Hint 2 — The plan, and almost the whole thing](#)
- [Solution](#)

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — What you're building, and the exact pieces

Basic Version

You're building three small files that work together: one that describes the prompt, one that builds the chain, and one that compares the chain to Project 1's raw version.

The chain's job: take the same input Project 1's raw feature takes, and produce the same kind of structured output — just built with `|` instead of raw API calls.

The comparison script's job: run both versions on the same inputs and prove, not just claim, that they agree.

Things to look up and use: - `ChatPromptTemplate.from_template(...)` — build your prompt in `prompts.py` - `.with_structured_output(YourModel)` — attach the same Pydantic shape Project 1 already uses. - `|` — chain the prompt and the structured model together. - Your `Doc01.load_config()` — read the model name and any settings from there, not hardcoded. - Project 1's raw extraction function — import it directly into `compare_with_raw_sdk.py`, don't rewrite it.

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

You're building two separate, reusable things that get used together: a `prompts.py` module holding the prompt template(s), and a `chain.py` module that builds and returns the finished `Runnable`. Keep them separate — later documents will import `chain.py`'s function without caring how the prompt itself is written.

The key requirement to hold onto: the model name and temperature must come from your Doc01 config, never hardcoded. This means `build_extraction_chain()` should call `load_config()` (or accept a config object as a parameter) rather than writing `ChatOpenAI(model="gpt-4o-mini")` directly inside it.

Here's what to actually go look at: - `prompts.py` should hold a function like `get_extraction_prompt() -> ChatPromptTemplate`, not a bare module-level variable — this makes it easier to test and to swap later without touching `chain.py`. - `chain.py`'s `build_extraction_chain() -> Runnable` should call `load_config()` from Doc01 (or accept a `Config` object as a parameter — decide which and be consistent). - **Reusing Project 1's Pydantic model**, not redefining it, is what actually makes the comparison meaningful. - `compare_with_raw_sdk.py` should print a clear pass/fail per test input, and print a clear summary line if anything mismatched.

Sketch, in plain English, the exact function signature of `build_extraction_chain()` before moving to Hint 2.

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Think past "make the comparison pass once." Ask: **what happens the next time someone changes the prompt wording, and who finds out if the LCEL chain and the raw version have quietly drifted apart?**

The answer that matters for a real codebase: `compare_with_raw_sdk.py` shouldn't just print pass/fail to the terminal for a human to read — it should be structured so it *could* become an actual automated test with minimal changes (Doc13 covers this properly). That means: collect results into a typed structure instead of printing as you go, and have the script exit with a clear signal (a non-zero status, or a final summary a CI system could grep for) rather than relying on someone reading scroll-back.

There's also a design decision hiding in `build_extraction_chain(config=None)`: should the function load its own config by default (convenient to call), or should config always be loaded once and passed down explicitly (every function's dependencies visible in its signature, easier to test with a fake config, no hidden I/O)? Both are used in real codebases — sketch both signatures before checking the Solution.

Difference between Basic, Intermediate, and Advanced: Basic names the 3 files and the tools each needs. Intermediate adds the real function signatures and the "config from `load_config()`, never hardcoded" constraint. Advanced asks what happens *after* this passes once — how the comparison script should be shaped so a future drift between the two versions actually gets caught automatically, and which of two real config-passing designs to commit to.

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and almost the whole thing

Basic Version

```
prompts.py:
    function get_extraction_prompt():
        return a ChatPromptTemplate with the same wording as Project 1's raw prompt

chain.py:
    function build_extraction_chain():
        load config (for model name, temperature)
        get the prompt from prompts.py
        build a ChatOpenAI with the config's settings
        attach with_structured_output using Project 1's Pydantic model
        return prompt | structured_model

compare_with_raw_sdk.py:
    import Project 1's raw function
    import build_extraction_chain

    make a list of test inputs
    chain = build_extraction_chain()

    for each input:
        raw_result = call the raw function
        lcel_result = call chain.invoke
        print both, print whether they match
```

The trickiest part — wiring config into the chain instead of hardcoding it:

```
from config import load_config
from langchain_openai import ChatOpenAI

def build_extraction_chain():
    config = load_config()
    model = ChatOpenAI(model=config.model_name, temperature=config.temperature)
    # ... attach prompt and structured output here
```

Add a `model_name` and `temperature` field to your Doc01 `Config` if it doesn't already have them.

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

```
from config import load_config, Config
from langchain_core.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI
from langchain_core.runnables import Runnable

def build_extraction_chain(config: Config | None = None) -> Runnable:
    if config is None:
        config = load_config()
    model = ChatOpenAI(model=config.model_name, temperature=config.temperature)
    # ... attach the prompt template and .with_structured_output(...) here
```

Notice the `config: Config | None = None` parameter — this lets tests pass in a fake config without touching real environment variables, while normal callers can just call `build_extraction_chain()` with no arguments.

Try finishing the rest yourself before looking at the full Solution below.

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Sketch `compare_with_raw_sdk.py`'s result-collection shape before writing the full script:

```
from dataclasses import dataclass

@dataclass
class ComparisonResult:
    input_text: str
    raw_result: "ExtractedData"
    lcel_result: "ExtractedData"
    matched: bool
```

Then write `run_comparison(test_inputs) -> list[ComparisonResult]` as a separate function from `main()` — this is what lets the exact same logic be called from a script's `if __name__ == "__main__":` block *and* from a future `pytest` test, without duplicating the comparison loop. `main()` should just call `run_comparison()`, then decide how to report the results (print, or `assert` if this becomes a real test).

Try writing both before checking the Solution.

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Solution

Two working solutions below. Both are correct — they show two normal, real ways people build this. Read both, and think about which one you'd actually pick and why.

Basic Version

Approach 1 — one flat `chain.py`

```
# prompts.py
from langchain_core.prompts import ChatPromptTemplate

def get_extraction_prompt():
    return ChatPromptTemplate.from_template(
        "Extract structured data from the following text: {text}"
    )
```

```
# chain.py
from config import load_config
from langchain_openai import ChatOpenAI
from prompts import get_extraction_prompt
from project_1 import ExtractedData  # reuse Project 1's Pydantic model

def build_extraction_chain():
    config = load_config()
    prompt = get_extraction_prompt()
    model = ChatOpenAI(model=config.model_name, temperature=config.temperature)
    structured_model = model.with_structured_output(ExtractedData)
    return prompt | structured_model
```

```
# compare_with_raw_sdk.py
from chain import build_extraction_chain
from project_1 import extract_raw

test_inputs = ["input one", "input two", "input three"]
chain = build_extraction_chain()

for text in test_inputs:
    raw_result = extract_raw(text)
```

```
lcel_result = chain.invoke({"text": text})  
print(text, "-> match:", raw_result == lcel_result)
```

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — config passed as an optional parameter

chain.py

```
from langchain_core.runnables import Runnable
from langchain_openai import ChatOpenAI

from config import Config, load_config
from prompts import get_extraction_prompt
from project_1 import ExtractedData

def build_extraction_chain(config: Config | None = None) -> Runnable:
    if config is None:
        config = load_config()

    prompt = get_extraction_prompt()
    model = ChatOpenAI(model=config.model_name, temperature=config.temperature)
    structured_model = model.with_structured_output(ExtractedData)

    return prompt | structured_model
```

compare_with_raw_sdk.py

```
from dataclasses import dataclass

from chain import build_extraction_chain
from project_1 import extract_raw, ExtractedData

@dataclass
class ComparisonResult:
    input_text: str
    raw_result: ExtractedData
    lcel_result: ExtractedData
    matched: bool

def run_comparison(test_inputs: list[str]) -> list[ComparisonResult]:
```

```

chain = build_extraction_chain()
results = []
for text in test_inputs:
    raw_result = extract_raw(text)
    lcel_result = chain.invoke({"text": text})
    results.append(ComparisonResult(text, raw_result, lcel_result, raw_result))
return results

def main() -> None:
    test_inputs = ["input one", "input two", "input three"]
    results = run_comparison(test_inputs)

    matched = sum(1 for r in results if r.matched)
    print(f"{matched}/{len(results)} matched")

    for r in results:
        if not r.matched:
            print(f"MISMATCH on {r.input_text!r}: raw={r.raw_result} lcel={r.lcel_result}")

if __name__ == "__main__":
    main()

```

Difference from Basic: Basic gets the comparison working with plain per-line prints. Intermediate separates `run_comparison()` (the logic) from `main()` (the reporting), and collects results into a typed `ComparisonResult` list instead of printing inline — this is what makes the script reusable as an actual test later, not just a one-off script.

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Approach 1 — config loaded once at the entry point, passed down explicitly

This version loads config exactly once, in `main()`, and passes it down through every function that needs it — instead of letting `build_extraction_chain()` call `load_config()` internally. Some teams prefer this because it makes every function's dependencies fully visible in its signature, with no hidden global state or hidden I/O buried inside a "build" function.

```
# chain.py
def build_extraction_chain(config: Config) -> Runnable:
    prompt = get_extraction_prompt()
    model = ChatOpenAI(model=config.model_name, temperature=config.temperature)
    return prompt | model.with_structured_output(ExtractedData)
```

```
# compare_with_raw_sdk.py
def main() -> None:
    config = load_config()
    chain = build_extraction_chain(config)
    results = run_comparison(chain, test_inputs=["input one", "input two", "input three"])
    matched = sum(1 for r in results if r.matched)
    print(f"{matched}/{len(results)} matched")
    if matched != len(results):
        raise SystemExit(1)  # non-zero exit: a CI system can detect this failed
```

Approach 2 — turned into a real `pytest` test

```
# test_chain_matches_raw.py
import pytest
from chain import build_extraction_chain
from project_1 import extract_raw

TEST_INPUTS = ["input one", "input two", "input three"]

@pytest.mark.parametrize("text", TEST_INPUTS)
def test_chain_matches_raw_sdk(text):
    chain = build_extraction_chain()
    assert chain.invoke({"text": text}) == extract_raw(text)
```

This is the real payoff of Intermediate's `run_comparison() / main()` split and the typed `ComparisonResult`: the exact same comparison logic, reshaped as `pytest` parametrized assertions, runs automatically in CI on every change — instead of relying on someone remembering to run `compare_with_raw_sdk.py` by hand before merging.

Difference from Intermediate: Intermediate makes the comparison script reusable in *shape*. Advanced actually reuses it two ways — Approach 1 makes the script itself CI-friendly (explicit config, a real exit code), and Approach 2 turns the same underlying comparison into an automated test that runs without anyone remembering to.

Which one should you actually use?

Basic's flat version is fine for getting the exercise working. Intermediate's optional-config-parameter pattern is the standard real-world default — convenient to call, still testable. Move to Advanced's explicit-config-plus-pytest version once this Reusable Chain Module is something other code actually depends on (it is — Project 2 onward reuses it) — at that point, a script a human has to remember to run by hand isn't enough; a real automated test that runs in CI is what actually catches the drift this exercise is about.